

INF-73625 Programación Avanzada para Ciencias

Tarea 2: Análisis de partículas mediante estructuras dinámicas

Profesores

- Ricardo Salas Letelier (ricardo.salas@usm.cl)
- Wladimir Ormazabal (wladimir.ormazabal@usm.cl)

Ayudantes

- Matías Gómez (mgomez@usm.cl)

Fecha de entrega: 25 de septiembre del 2026

Índice

1. Reglas	2
2. Objetivos	2
3. Problemas a Resolver	2
3.1. Análisis de partículas mediante estructuras dinámicas	2
3.2. Construcción del Árbol Binario de Búsqueda	3
3.3. Selección y construcción de la lista enlazada	3
3.4. Colapso Energético (Eliminación Condicional)	4
3.5. Inversión Magnética de Partículas	4
4. Entrega de la Tarea	4
5. Restricciones y consideraciones	5
6. Documentación	5

1. Reglas

La presente tarea debe hacerse de forma **individual**. Toda excepción a esta regla debe ser conversada con los ayudantes (usando los correos indicados en el encabezado). No se permiten de ninguna manera trabajos en grupo.

Debe usarse el lenguaje de programación C++. Al evaluarla, la tarea será compilada usando el compilador `g++`, con la línea de comando `g++ archivo.cpp -o output -Wall`. Alternativamente, se aceptan variantes como MinGW.

Todas las estructuras de datos solicitadas deben ser implementadas completamente por cada estudiante. **No se permite usar la biblioteca `stl`**, así como ninguno de los `containers` y algoritmos definidos en ella. Está permitido usar bibliotecas estándar como `math.h`, `string`, `fstream`, `iostream`.

2. Objetivos

Entender y familiarizarse con la implementación y uso de estructuras de datos de tipo listas y árboles. Además, se fomentará el uso de buenas prácticas y el orden en la programación.

3. Problemas a Resolver

Entregue un archivo `.cpp` con una función `main` adecuada que permita probar su programa.

3.1. Análisis de partículas mediante estructuras dinámicas

En un experimento de física de partículas se registran distintos eventos en un detector. Dada la cantidad de información generada, se desea gestionar esta a través de estructuras de datos, donde se utilizará un Árbol Binario de Búsqueda (ABB) para almacenar los registros y una lista enlazada para las partículas que cumplan ciertas condiciones.

Cada partícula estará caracterizada por:

- **ID:** identificador único (string).
- **Energía:** (float).
- **Masa:** (float).

El ABB deberá estar ordenado de acuerdo con la energía de las partículas (se garantiza que no habrá dos partículas con la misma energía).

Formato de entrada

Se entregará un archivo `Particulas.txt`. La primera línea contendrá un entero N (cantidad de partículas). A continuación, se tendrán N líneas con la información en el formato: `ID ENERGIA MASA`. Su programa deberá ser capaz de abrir este archivo y leer los datos para poblar las estructuras.

Formato `Particula.txt`

```
1 8
2 P001 12.5 0.511
3 P002 7.8 1.20
4 P003 18.4 2.35
5 P004 4.2 0.105
6 P005 15.7 0.938
7 P006 22.1 4.18
8 P007 9.6 0.139
9 P008 13.3 0.494
```

3.2. Construcción del Árbol Binario de Búsqueda

Los registros deberán almacenarse en un ABB, utilizando la energía como criterio de orden. Implemente:

```
1 void insertar(string id, float energia, float masa);
```

- **Entrada:** El ID de la partícula, su energía y su masa (leídos del archivo txt).
- **Resultado:** Modifica la estructura interna del árbol añadiendo un nuevo nodo.

Ejemplo de uso: Al invocar `insertar("P001", 12.5, 0.511);`, la partícula se ubica estructuralmente en el ABB según su valor de energía.

Además, se deberá implementar un recorrido del árbol que permita visualizar la información almacenada en él. Implemente la función:

```
1 void preOrden();
```

- **Resultado:** Imprime en consola un recorrido en **pre-orden** (raíz, izquierda, derecha) con la información completa de cada nodo, respetando el formato del archivo original.

Ejemplo de salida esperada:

```
1 P001 12.5 0.511
2 P002 7.8 1.2
3 P004 4.2 0.105
4 P007 9.6 0.139
5 P003 18.4 2.35
6 P005 15.7 0.938
7 P008 13.3 0.494
8 P006 22.1 4.18
```

3.3. Selección y construcción de la lista enlazada

Para motivos del experimento, el usuario deberá definir dinámicamente un valor U (umbral de energía). Se recorrerá el ABB seleccionando partículas con energía $\geq U$, incorporándolas a una lista enlazada con nodos independientes. Implemente:

```
1 void insertarAlFinal(string id, float energia, float masa);
```

- **Entrada:** El ID, energía y masa de la partícula extraída del árbol.
- **Resultado:** Crea un nuevo nodo independiente y lo enlaza al final de la lista.

Ejemplo de uso: Al invocar `insertarAlFinal("P006", 22.1, 4.18);`, se instancia un nuevo nodo que se conecta al final de la estructura.

Una vez construida la lista, se deberá recorrer de manera secuencial e imprimir su contenido desde el inicio hasta el final. Implemente la función:

```
1 void imprimirLista();
```

- **Resultado:** Imprime en consola **únicamente el ID** de las partículas separadas por flechas.

Ejemplo de salida esperada:

```
1 P006 -> P003 -> P005 -> NULL
```

3.4. Colapso Energético (Eliminación Condicional)

Tras aplicar los campos magnéticos, los físicos notaron que si dos partículas adyacentes en la lista concentran demasiada energía, la interacción se vuelve inestable. Si la suma de la energía de una partícula y la de su sucesora inmediata supera un límite E_{max} , la partícula con **mayor energía** entre ambas colapsa y debe ser eliminada del registro.

Implemente la función:

```
1 void purgarInestabilidad(float eMax);
```

- **Entrada:** El límite de energía máxima permitida entre dos nodos adyacentes (**eMax**).
- **Resultado:** Recorre la lista evaluando pares de nodos conectados. Si su energía combinada supera **eMax**, elimina el nodo con mayor energía usando **delete** y reconecta los punteros **siguiente** correctamente para mantener la integridad de la lista.

Ejemplo de funcionamiento: Suponga que se define un umbral de inestabilidad **eMax** = 30.0 y la lista actual contiene las siguientes energías:

```
1 P006(E: 22.1) -> P003(E: 18.4) -> P005(E: 15.7) -> NULL
```

Evaluación algorítmica:

- Se evalúa el par P006 y P003: $22,1 + 18,4 = 40,5$. Supera el límite de 30.0. Como P006 tiene mayor energía (22.1 ¿18.4), colapsa y es eliminado. La nueva cabeza pasa a ser P003.
- Se evalúa el nuevo par adyacente P003 y P005: $18,4 + 15,7 = 34,1$. Supera el límite de 30.0. Como P003 tiene mayor energía (18.4 ¿15.7), colapsa y es eliminado.

Tras invocar **purgarInestabilidad(30.0)**, una llamada a **imprimirLista()** mostrará:

```
1 P005 -> NULL
```

3.5. Inversión Magnética de Partículas

Durante el análisis de las partículas seleccionadas en la lista, el detector fue expuesto accidentalmente a un campo magnético inverso severo. Como consecuencia, el orden cronológico de las partículas en la lista enlazada quedó exactamente opuesto. Implemente la función:

```
1 void InvertirLista();
```

- **Resultado:** Reasigna los punteros internos de los nodos para que el orden de la lista se invierta físicamente.

Ejemplo de funcionamiento: Si la lista original era:

```
1 P006 -> P003 -> P005 -> NULL
```

Después de llamar a esta función, una llamada a **imprimirLista()** deberá mostrar:

```
1 P005 -> P003 -> P006 -> NULL
```

4. Entrega de la Tarea

Entregue la tarea enviando un archivo comprimido **tarea2-nombre-apellido.zip** (reemplazando por nombre y apellido según corresponda) a la página **aula.usm.cl** del curso, a más tardar el día 28 de septiembre 2026, a las 23:59 (Chile Continental), el cual contenga:

- Los archivos con los códigos fuentes necesarios para el funcionamiento de la tarea. ¡Los archivos deben compilar!

- **README.txt:** Instrucciones de compilación en caso de ser necesarias, y la forma de compilación que usó (debe ser alguna de las indicadas en los tutoriales entregados en Aula).

5. Restricciones y consideraciones

- -10 pts por cada día o fracción de retraso
- Por cada Warning, -5 puntos.
- Las tareas que no compilen serán calificadas con nota 0.
- Si se detecta COPIA la nota automáticamente será 0.
- Toda memoria reservada dinámicamente con **new** debe ser liberada con **delete** o **delete[]** antes de finalizar el programa.
- La falta de orden será penalizada

6. Documentación

El código fuente del programa **debe** estar estructurado adecuadamente en archivos (separados de ser necesario). Si el código fuente está desordenado, se descontarán puntos de la nota.

Las funciones **deben** estar comentadas con el siguiente formato:

```

1  /*****
2  * TipoFuncion NombreFuncion
3  *****/
4  * Resumen Funcion
5  *****/
6  * Input:
7  * tipoParametro NombreParametro : Descripcion Parametro
8  * .....
9  *****/
10 * Returns:
11 * TipoRetorno, Descripcion retorno
12 *****/

```